

FAIT Gadgets E-Store

How the entire site works — every input, every output, every button, every app it depends on.

Live store: fait-gadgets-estore.vercel.app · Admin login: [/admin](#)

This is the single reference for how the entire site works. It is written for the store owner, not just a developer. If a word is unfamiliar, check the **Glossary** near the end.

Related documents in this project, and how this manual differs from them:

- [CLAUDE.md](#) — technical reference for whoever is coding on this project (architecture, file locations, rules). Read this manual instead if you just want to know how to *use* the site.
- [SESSION_LOG.md](#) — a diary of what was built/fixed in each work session, in order.
- [SETUP_GUIDE.md](#) — **outdated**. Still describes Green API and CallMeBot, both removed long ago. Don't follow it for new setup; this manual and [CLAUDE.md](#) 's Environment Variables section are current.

Contents

- | | |
|--|-----------------------------------|
| 1. The Big Picture | 7. Environment Variables |
| 2. The Two Ways to List a Product | 8. Troubleshooting |
| 3. Managing Products — the Admin Dashboard | 9. Local Development & Deployment |
| 4. What Customers See and Can Do | 10. The Database, Simplified |
| 5. Notifications | 11. Glossary |
| 6. Apps and Services This Project Depends On | 12. Quick Reference Cheat Sheet |

1. The Big Picture — how a product gets from your phone to a customer's screen



Everything else in this manual is detail on top of this one loop: **you provide a photo and maybe some words → AI fills in the rest → it appears on the store → customers act on it → you get told about it.**

2. The Two Ways to List a Product

WhatsApp (via Meta's Cloud API) was the original plan, but Meta rejected the business verification needed to make it work — so it was removed from the project entirely. Telegram fully replaced it: it lists products AND carries every seller notification, at zero cost with no verification risk.

Channel	Status	Use it when
Telegram bot	✅ Live, primary	Always — it's the fastest, works from your phone, zero setup per item
Admin Upload page (/admin/products/new)	✅ Live, backup	Telegram is down, or you want to carefully review every field before it goes live (Draft mode)

2.1 Telegram Bot — the main way you list products

Find your bot in Telegram (the one whose token is in `TELEGRAM_BOT_TOKEN`) and just message it.

To list ONE product with one photo:

1. Open a chat with the bot in Telegram
2. Send a photo, with a caption describing it (e.g. `Red leather handbag ₦8,500, brand new`)
3. Wait a few seconds — the bot replies with a confirmation and a link to the live listing
4. The product is **published immediately** (not a draft) — it's already live on the store

To list ONE product with MULTIPLE photos (an album):

1. In Telegram, tap the photo icon and **select several photos at once** (this creates what Telegram calls an "album" / "media group")
2. Only ONE of the photos needs a caption — Telegram lets you add a caption to the whole album, usually on the first photo you select
3. Send them all together
4. Wait a few seconds longer than for a single photo (the bot waits ~2 seconds after the last photo arrives to make sure it has them all) — then you get ONE confirmation message for ONE product, with all the photos attached
5. **Important:** put your price and description details in the caption. If a photo arrives without ever getting a caption attached to the album, the AI only has the photo to go on and may not catch the price.

If you made a mistake (wrong price, name, etc.) after sending:

- **Do NOT edit the message/caption in Telegram** — this does nothing. Telegram doesn't tell the bot when you edit a message, so the listing never updates.
- **Instead, send a brand new text message** describing the fix. Examples: `price 165000` , `change the price to ₦165,000` , `update the name to Samsung Galaxy Buds Pro` , `change category to Electronics` .
- This always applies to whichever product you listed **most recently** (regardless of whether it's still live, sold, or anything else).
- The bot will ALWAYS reply — either confirming what it changed, or telling you it didn't understand and giving you an example of what to try. It is never silent.
- If you need to fix something that ISN'T your most recent listing, use the **Admin Dashboard → Edit** instead (see Section 3).

To mark your most recent LIVE (available) product as sold:

- Send the exact text `SOLD` (any case — `sold` , `Sold` , `SOLD` all work)
- The bot replies confirming which product it marked sold

Things the Telegram bot does NOT do:

- It can't tell you what your current listings are (no "list my products" command) — check the live site or the Admin Dashboard for that.
- It can't delete a product — use the Admin Dashboard.
- It can't mark a *specific* product as sold by name if you've listed several since — `SOLD` always targets the most recent available one.

2.2 Admin Upload Page — the backup way, with more control

Go to `/admin` → log in → **Add Product**.

1. **Tap to take a photo, or pick from your gallery** — or tap "pick from your image library" to reuse a photo you've already uploaded for a different product
2. Wait while the AI reads it (spinner shown)
3. Review the pre-filled fields — **name, price + currency (NGN/USD), category, description** — edit anything that's wrong
4. Optionally add up to 5 more photos (tap **+** to upload a new one, or the grid icon to pick from your library) — only the **FIRST** photo gets read by the AI; extra photos are just stored
5. Choose:
 - **Save as Draft** — saved but invisible to customers, until you publish it later
 - **Publish Now** — goes live immediately

This is the only channel that supports **Draft** mode (review before going live). Telegram listings always publish immediately.

3. Managing Products — the Admin Dashboard

Go to `/admin/dashboard` (log in first if needed).

3.1 What you see

- **Stats row:** count of Live / Drafts / Sold products
- **+ Add Product** button → takes you to the upload page (Section 2.2)
- **Three tabs:** Live, Drafts, Sold — each lists the matching products

3.2 Actions available per product, by status

Status	Buttons available	What each does
Draft	Edit,  Publish, Delete	Publish makes it instantly visible on the store
Live	Edit, Unpublish, Mark Sold, Copy URL, Delete	Unpublish sends it back to Drafts (invisible to customers, but not deleted)
Sold	Edit, Re-list, Delete	Re-list sets status back to Live/available

- **Edit** → opens a full-page editor where you can change name, price, currency, category, description, and add/remove/reorder photos (same photo tools as the upload page — new upload, or pick from your image library)
- **Delete** → permanent. Asks for confirmation first. Cannot be undone.
- **Copy URL** → copies the public product page link to your clipboard, ready to paste anywhere (e.g. to share directly with a buyer)

3.3 How fast do changes appear on the live store?

Instantly. Every admin action (publish, edit, delete, status change) immediately tells the live site to refresh that page — you don't need to wait or do anything extra. (Telegram listings/edits do the same.) There's a 10-minute fallback refresh in case that instant signal is ever missed, but in practice you should see changes appear right away.

4. What Customers See and Can Do

4.1 Browsing the storefront

- **Category filter** — pills along the top: All, Fashion, Electronics, Food & Drinks, Beauty, Home & Living, Other
- **Live search** — typing in the search box filters the grid as they type (no page reload), searching by product name
- **Dark mode toggle** — a small sun/moon button in the header. Switches the whole site to a dark color theme. Their choice is remembered on their device for next time.

4.2 On a product page

- **Photo gallery** — large image with a thumbnail strip if there are multiple photos
- **Like** (heart icon) and **Save** (bookmark icon) — these notify you (see Section 5)
- **"Request to Buy"** button — opens a form where the customer enters their name, phone number, and an optional message. Submitting this saves their contact details, notifies you immediately, and shows the customer a confirmation that you'll reach out
- **Chat bubble icon** — opens the Tawk.to live chat widget for general questions (separate from the Request to Buy flow)
- **SOLD overlay** — if you've marked the item sold, the page still works (doesn't 404) but shows a "This item has been sold" message instead of the buy button

4.3 What customers can't do

- They cannot see Draft products at all (the page returns "not found")
- They cannot edit, delete, or directly negotiate a price through the site — that all happens via the Request to Buy form or the chat widget, with you replying personally

5. Notifications — what tells you something happened, and how

Every notification goes through one channel: **Telegram**, the same chat the bot lists products in.

Event	Notified via	Note
New product listed (Telegram)	Telegram message, same chat	Includes name, price, link
New product listed (Admin Upload, published)	<i>(none — you just did it yourself)</i>	
Someone likes a product	Telegram message	
Someone saves a product	Telegram message	
Someone submits a "Request to Buy"	Telegram message	Includes buyer's name, phone, and optional message
Someone marks SOLD via Telegram	Telegram message	Confirms which product
Someone sends a free-text edit instruction	Telegram message	Confirms the change, or explains it didn't understand

Views (just looking at a product) are never notified — only likes, saves, and buy requests.

6. The Apps and Services This Project Depends On

Service	Role	Where to log in	If it breaks...
Supabase	The database — stores every product, interaction, enquiry	supabase.com	Site can't show or save products at all
Cloudinary	Stores and serves all product photos	cloudinary.com	New photos fail to upload; old photos may stop loading if account is suspended
Google Gemini AI	Reads photos + captions/instructions, fills in product details, interprets Telegram text edits	aistudio.google.com	AI fields stop pre-filling; Telegram text edits stop working
Telegram Bot API	The listing channel AND every seller notification	t.me (talk to your own bot)	No new listings via Telegram, and no notifications at all; admin upload still works as backup for listing
Tawk.to	Live chat widget on the storefront	tawk.to (also has a phone app)	Chat bubble disappears; Request to Buy form still works independently
Vercel	Hosts the live website, auto-deploys on every <code>git push</code>	vercel.com	Site goes offline entirely
GitHub	Stores all the code, triggers Vercel deploys	github.com	Can't deploy new changes (site keeps running on its last deploy)

None of these cost money at the usage levels this store generates (all free tiers).

7. Environment Variables — what they're for

These are the secret settings that connect the code to the services above. Actual current values live in `.env.local` (on this computer, never committed to GitHub) and must be duplicated in the Vercel dashboard (Settings → Environment Variables) for the live site to work. See `CLAUDE.md` for the exact current values.

Variable	What it's for
<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>	Database connection
<code>CLOUDINARY_CLOUD_NAME</code> , <code>CLOUDINARY_API_KEY</code> , <code>CLOUDINARY_API_SECRET</code>	Photo storage
<code>GEMINI_API_KEY</code>	AI photo/text reading
<code>TELEGRAM_BOT_TOKEN</code> , <code>TELEGRAM_CHAT_ID</code>	The Telegram bot's identity and where it sends you every notification
<code>NEXT_PUBLIC_STORE_NAME</code>	Shown as the site title
<code>NEXT_PUBLIC_SITE_URL</code>	Used to build links sent in notifications
<code>NEXT_PUBLIC_TAWKTO_ID</code>	Connects the live chat widget to your Tawk.to account
<code>ADMIN_PASSWORD</code>	The password for <code>/admin</code>

Gotcha to remember: if any value contains a `#` character (like a password), it MUST be wrapped in quotes in `.env.local` — e.g. `ADMIN_PASSWORD="abc#123"` — otherwise everything after the `#` is silently cut off when the site loads locally. This already happened once (see Section 8). Vercel's own dashboard doesn't have this problem; it only affects local testing on your computer.

8. Troubleshooting — common problems and exactly what to do

Symptom	Likely cause	Fix
Admin password "wrong" locally, but you're sure it's right	A <code>#</code> in the password got truncated by the local env loader	Quote the value in <code>.env.local</code> : <code>ADMIN_PASSWORD="yourpass#123"</code> , restart <code>npm run dev</code>
One Telegram album created several separate products	You're on an old deploy from before the album fix	Make sure <code>schema.sql</code> was run in Supabase (adds <code>telegram_media_groups</code> table) and the latest code is deployed
Edited a Telegram caption and nothing changed	Telegram doesn't notify the bot about edits at all	Send a brand-new text message instead, e.g. <code>price 165000</code>
Sent a text message to the bot and nothing happened	Old code, or Gemini genuinely couldn't understand the message	Update to latest code; rephrase more explicitly, e.g. <code>change the price to 50000</code>
A product shows "Ask for price" / no price	AI couldn't find a price in the photo or caption	Send a Telegram text edit (<code>price 50000</code>) or use Admin → Edit
New product doesn't appear on the storefront	Could be saved as Draft, not Published	Check Admin Dashboard → Drafts tab; Publish it if found there
Photos won't upload from Admin or Telegram	Cloudinary credentials wrong/expired, or file too large	Check Cloudinary dashboard is active; re-check <code>CLOUDINARY_*</code> env vars
AI fields come back empty or wrong on every upload	Gemini API key invalid/expired, or rate-limited	Check aistudio.google.com for key status; form still lets you fill fields manually if AI fails
Telegram bot stops responding entirely	Bot token revoked, or webhook unregistered	Re-run the <code>setWebhook</code> URL (see <code>CLAUDE.md</code> → How to Deploy) with the current token
Stopped getting ANY notifications (like/save/buy-request/listing)	Telegram bot token revoked, or the bot was blocked/removed from the chat	Message the bot again to confirm it responds; re-check <code>TELEGRAM_BOT_TOKEN</code> / <code>TELEGRAM_CHAT_ID</code> are still correct

<code>vercel --prod</code> logs into the wrong account	Known unresolved issue	Skip the CLI: just <code>git push</code> to <code>main</code> , Vercel's GitHub integration auto-deploys
Pushing to GitHub fails / wrong account	<code>gh auth status</code> shows the wrong account	Run <code>gh auth switch --user FAIT-Pro</code>
Site shows old content after a deploy	Browser cache, or the deploy hasn't finished yet	Hard-refresh; check vercel.com dashboard for deploy status
Cannot find module './XXX.js' error locally, or buttons/clicks stop doing anything in the dev server	Stale Next.js build cache — often caused by running a production <code>npm run build</code> while <code>next dev</code> is still running against the same <code>.next</code> folder	<code>lsof -ti:3000 xargs kill -9 && rm -rf .next && npm run dev</code> (use the correct port if not 3000)
Sold item shows a 404 instead of a SOLD badge	Would be a regression of a previously-fixed bug	Should not happen on current code — check the product page uses the admin database client, not the public one

General debugging tip: Vercel's dashboard (vercel.com → your project → a deployment → "Functions" tab) shows server-side log output for any API route, including the Telegram and webhook handlers. This is the first place to look when something fails silently in production.

9. Local Development & Deployment — for whoever is coding on this

```
# Install dependencies (first time only)
npm install

# Run locally
npm run dev
# → http://localhost:3000

# Check for type errors before trusting a change
npx tsc --noEmit

# Build exactly as production would
npm run build

# Commit and deploy
git add <files>
git commit -m "... "
git push          # Vercel auto-deploys main on every push
```

Before any major change, create a snapshot branch first (this project's convention):

```
git checkout -b snapshot-vN-description
git push origin snapshot-vN-description
git checkout main
```

At the start of any work session, confirm accounts are correct:

```
gh auth status          # should show FAIT-Pro as active
gh auth switch --user FAIT-Pro  # if not
vercel whoami           # often wrong – known unresolved issue, ignore and use git push
```

10. The Database, Simplified

Four tables, all in Supabase:

- products** — every item ever listed. **status** is one of **available** (live), **draft**, or **sold**. **image_urls** holds ALL photos for multi-photo items; **image_url** is just the first one, kept for backward compatibility. **wa_message_id** is how the system avoids creating a duplicate product if a webhook delivery repeats itself.

- **interactions** — a log of every view/like/save/enquiry, used to power notifications and could be used for analytics later.
- **enquiries** — every "Request to Buy" submission, with the buyer's name, phone, and optional message.
- **telegram_media_groups** — a short-lived technical table that temporarily holds photos from a Telegram album while they're still arriving, before they get merged into one product. You'll never need to look at this directly; it cleans itself up automatically.

11. Glossary

Term	Meaning
Draft	A product saved but not visible to customers yet
Live / Available	Visible to customers, can be bought
Sold	No longer for sale; page still loads but shows a SOLD badge instead of the buy button
Webhook	A URL that another service (Telegram) calls automatically when something happens, e.g. when you send the bot a photo
Media group / album	Telegram's term for several photos sent together in one go
RLS (Row Level Security)	A database rule that controls who can see what — e.g. customers can only ever see available products, never drafts
Revalidate	The mechanism that makes the live site update instantly after an admin action, instead of waiting for a scheduled refresh
Env var (environment variable)	A secret setting (like a password or API key) kept outside the code, in .env.local locally and the Vercel dashboard in production
Deploy	Publishing a code change so it's live on the real website
Snapshot branch	A saved copy of the code at a point in time, used as a safety net before risky changes

12. Quick Reference — the most common things you'll actually do

I want to...	Do this
List a new product	Send a photo (or several at once) to the Telegram bot, with a caption including the price
Fix a price/name/description I got wrong	Send a NEW text message to the bot, e.g. <code>price 50000</code>
Mark something sold	Send <code>SOLD</code> to the bot
See everything I've listed	Go to <code>/admin/dashboard</code>
Carefully review before publishing	Use <code>/admin/products/new</code> → Save as Draft → review → Publish later
Edit a product that ISN'T my most recent listing	<code>/admin/dashboard</code> → find it → Edit
Delete a product	<code>/admin/dashboard</code> → find it → Delete (cannot be undone)
Share a product link directly with a buyer	<code>/admin/dashboard</code> → find it → Copy URL
Check if I'm missing any buy requests	<code>/admin/dashboard</code> (and check Telegram / Tawk.to for notifications)
Switch the site to dark mode	Tap the sun/moon icon in the header (works for you AND customers, separately)